Transition-Based vs Graph-Based Dependency Parsers

A practical trade-off study on English and Russian · UD 2.13 test sets

Aleksandr Gavkovskii
Innopolis University · NLP 2026 · Case Study 5.5

NLP 2026

Code & data

MOTIVATION

Transition-based (spaCy)
Shift-reduce arc-eager · greedy local decisions · O(n) · risk of error cascades

Graph-based (Stanza)
Biaffine attention + MST decoding · global tree · O(n²) · structurally consistent

- **Q1.** How large is the speed / accuracy trade-off in practice?
- **Q2.** Does morphological richness (Russian) amplify the gap?
- **Q3.** Where, structurally, does graph-based earn its wins?

EXPERIMENTAL SETUP

	spaCy	Stanza
Family	Transition-based	Graph-based
Architecture	shift-reduce (arc-eager)	biaffine + MST
EN model	en_core_web_trf (RoBERTa)	en pipeline
RU model	ru_core_news_lg (CNN)	ru pipeline

- **EN data:** UD English-EWT — 2,077 test sentences
- **RU data:** UD Russian-SynTagRus — 8,800 test sentences
- Gold tokenization · UAS + LAS (punct excluded)
- CPU tok/s (3 repeats + warmup) · peak memory via `tracemalloc`

WHERE GRAPH-BASED WINS STRUCTURALLY

Slice	spaCy	Stanza	Δ
Overall RU UAS	0.895	**0.936**	**+4.1 pt**
Long arcs RU (|h−d|>5)	0.742	**0.840**	**+9.8 pt**
Long arcs EN (|h−d|>5)	0.388	**0.760**	**+37.2 pt**
Non-projective arcs RU	0.321	**0.632**	**+31.1 pt**

+4.1 pt aggregate hides a +31 pt gap on non-projective arcs and +9.8 pt on long arcs — exactly where shift-reduce structurally cannot recover.

ALGORITHMIC METHODS

- **Shift-reduce (spaCy):** stack + buffer, arc-eager transitions, greedy O(n) — each decision is final, no backtracking
- **Biaffine + MST (Stanza):** scores all n² arc candidates with attention, then extracts the globally optimal tree via Chu-Liu/Edmonds
- **EN backbone:** RoBERTa transformer (spaCy) vs LSTM encoder (Stanza)
- **RU backbone:** CNN tagger (spaCy) vs LSTM encoder (Stanza)

CONCLUSIONS

- **Speed:** spaCy 17× faster RU, 2.5× EN. But uses 7.6× more peak memory on RU — if memory is bounded, the trade-off reverses.
- **Accuracy gap is real but narrow overall:** +4.1 pt UAS on RU; +29 pt on EN is genuine attachment error, confirmed by CLEAR→UD remap (not label bias).
- **Gap concentrates where theory predicts:** +31 pt on RU non-projective, +9.8 pt on long arcs. Global MST decoding does real work.
- **Morphological richness alone doesn't flip the verdict.** Effect concentrates on free-word-order structural properties (non-projectivity, long deps), not morphology per se.
- **Recipe:** High-volume short text → spaCy (17× throughput). Offline, long/literary Russian → Stanza (non-projective arcs will be mis-routed by shift-reduce). Always verify label scheme before trusting LAS.

CORE RESULTS

Parser	Lang	UAS	LAS	Tok/sec	Peak mem
spaCy (transition)	EN	0.615	0.450*	1,231	25.3 MB
Stanza (graph)	EN	**0.907**	**0.883**	488	**13.6 MB**
spaCy (transition)	RU	0.895	0.845	**12,435**	70.9 MB
Stanza (graph)	RU	**0.936**	**0.904**	729	**9.3 MB**

*spaCy EN emits CLEAR-style labels (dobj/pobj), not UD. After remap: LAS 0.557. Use UAS for fair EN comparison.

Speed: spaCy **2.5× faster EN, 17× faster RU**

Mem: Stanza **1.9× less EN, 7.6× less RU**

ACCURACY — UAS & LAS BY PARSER FAMILY

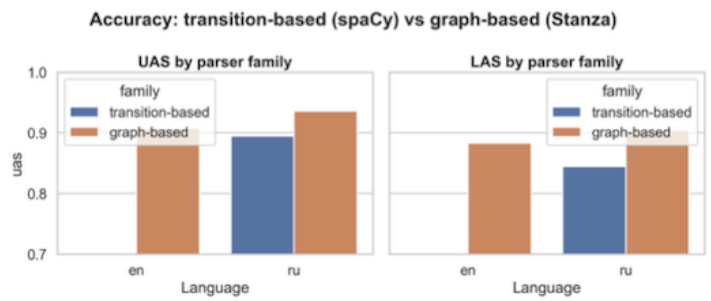

Fig 1. UAS and LAS on EN-EWT and RU-SynTagRus. Stanza leads UAS: +29 pt EN, +4.1 pt RU.

LONG-DISTANCE ARCS (|HEAD − DEP| > 5)

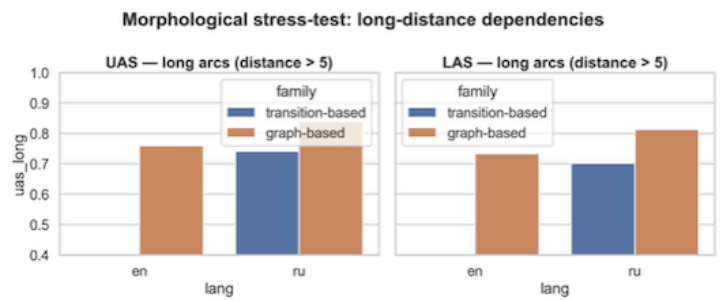

Fig 2. UAS on long arcs only. Gap doubles vs aggregate: +9.8 pt RU, +37.2 pt EN — where MST decoding matters most.

PROJECTIVITY SLICE — STRUCTURAL WEAK SPOT OF SHIFT-REDUCE

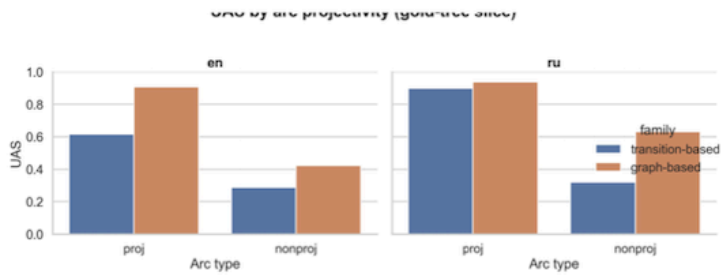

Fig 3. RU non-projective arcs: spaCy 0.321 vs Stanza 0.632 (**+31 pt**). Graph-based is nearly 2× better on constructions that violate projectivity.